

Project Astro Custom ISO Rollout Framework

Overview

The original Project Astro is a comprehensive integrated project management framework that handles server lifecycle, workload provision, inventory and billing all in a single pane of glass. All functions are tightly integrated and runs on python / FastAPI, bash, and Vue.js for a clean frontend.

Over the course of development of this project, the codebase has become large and hard to maintain as our PhD developers need to return to school to complete their studies.

And during review we have found a critical design issue, in which:

```
For our customer's security and compliance reasons, Mibura does not retain control
its infrastrcuture's Baseboard Management Controllers.
```

This invalidates all current assumptions on inventory management and automated provisioning workflows.

A New Proposal

Drawing inspiration from OpenStack's modular appraoch, and revolving around the constraint of no access to BMC hardware for provision, here is our new proposal to Project Astro:

```
A purpose-built autoprovision module that handles OS provision / re-provision
that:
- Integrates into customer workflows
- Customer input BMC credentials
- Can be operated entirely without Mibura involvement
- Easy to maintain and add features
- Single Drive and software RAID (mdadm) support
```

Specification

1. Custom iPXE ISO generation

- Writes target machine configuration into iPXE for static IP booting
- find interface that would connect to Internet using the configuration
- record the good interface's MAC address
- Fetch next-stage boot files, and handover to next stage

2. NEW: CustomOS

- Based on Ubuntu network install with Mibura's custom init script override
- Reports iPXE boot information back to configuration server
 - Retrieves hard drive information and send to configuration server
 - Configuration server decides hard drive installation via explicit UUID (single drive or RAID)
 - Default selection is smallest single drive / smallest two same-size drives for RAID

- Retrieves next stage boot information from configuration server
- fetch and boot into actual installer using information retrieved

3. Ubuntu Autoinstall

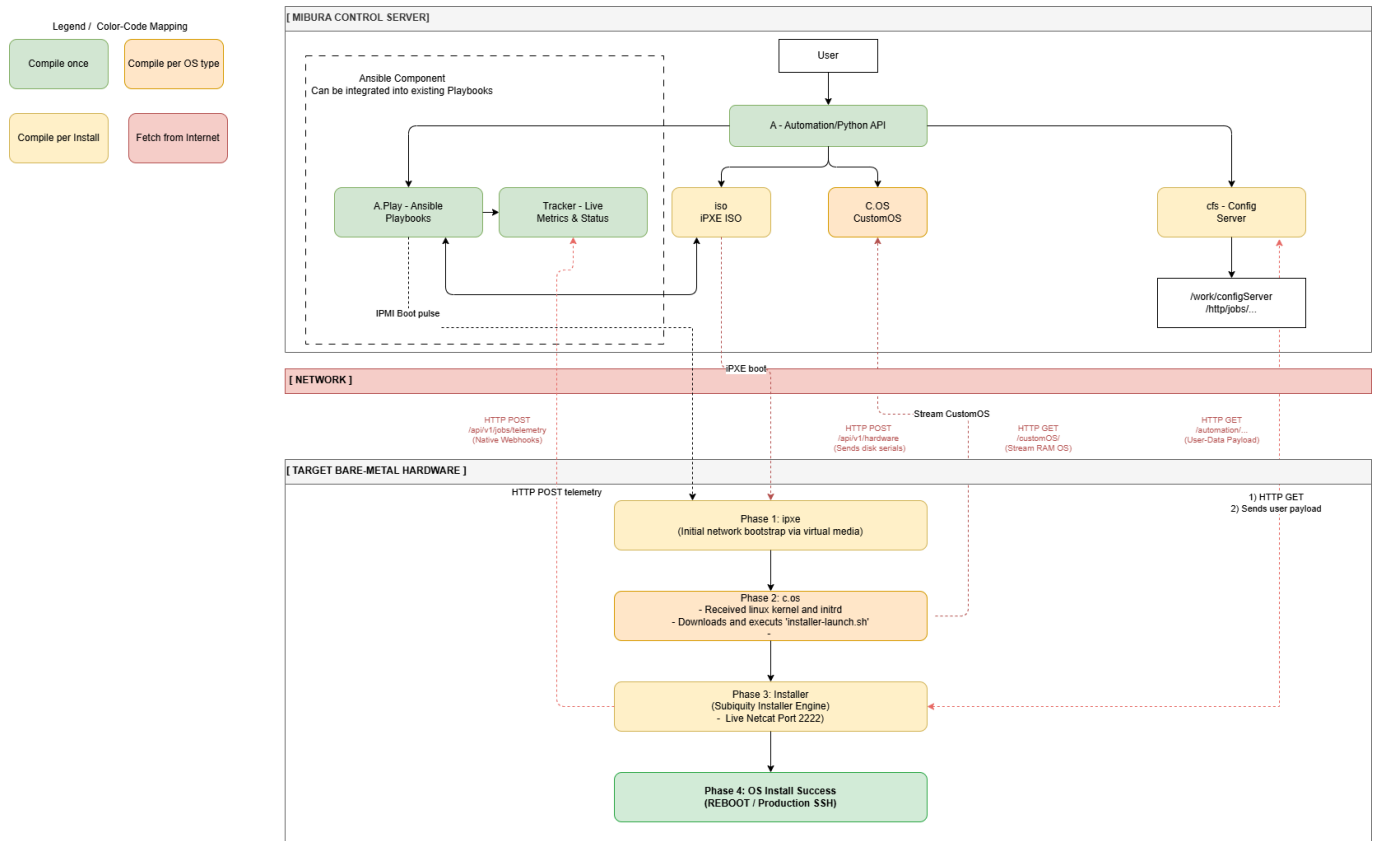
- Fetches information from network and from configuration server
- NEW: progress reporting to configuration server
- Automatically installs operating system

4. NEW: Ansible Playbooks for ISO insertion and one-time-boot

- Replaces Database + Python approach
- Using Ansible to call ISO generation API
- Redfish BMC operations
 - Insert retrieved ISO (URL) into BMC
 - Set One-Time-Boot
 - Restart machine to begin provision / reprovision

5. API:

- Ansible + custom ISO one-click deployment
 - Job status check
- custom ISO generation



Capabilities

1. End-To-End Provision (Ansible + custom ISO)

- Theoretically any Dell iDRAC9+ machines given:
 - reachable IP address
 - BMC username + password
- Tested: R440, R450, XR11
- Supermicro WTR server support coming 7/1/2026
- Cisco UCS will be tested once we receive them

2. customISO-only installation

- Any bare metal server that can take the ISO
- Additionally tested: proxmox virtual machines (minimum 11000MB ram required)

3. Tested and implemented OS:

- Ubuntu 22.04.5
- Ubuntu 24.04.2
- Ubuntu 24.04.3
- Ubuntu 24.04.4

4. RAID capabilities

- Single Disk installation (also for hardware RAID controllers)
- Software (mdadm) RAID-1

Hosting and building

We use a single docker container (and docker-compose) to provide all dependencies needed for deployment and build.

Deployment streamlining is still in-progress but it will be simplified to:

```
docker compose build
```

and

```
docker compose run -d
```

Example API Call (subject to change)

ISO Generation Only (IPV4 only example)

```
POST /api/v1/servers/provision/custom-iso

{
  "os_type": "ubuntu",
  "os_version": "24.04.4",
  "arch": "amd64",
  "variant": "ipxe-uefi",
  "hostname": "test",
  "username": "default",
  "password": "default",
  "ipv4_address": "10.1.10.142",
  "ipv4_gateway": "10.1.10.1",
  "ipv4_netmask": "255.255.255.0",
  "dns_servers": "8.8.8.8,8.8.4.4,1.1.1.1,2001:4860:4860::8888",
  "raid": false
}
```

Response:

```
{
  "success": true,
  "body": {
    "custom_id": "custom-test",
    "iso_url": "http://10.1.10.130:8000/automation/test/custom-test.iso",
    "user_data_url": "http://10.1.10.130:8000/automation/test/user-data",
    "meta_data_url": "http://10.1.10.130:8000/automation/test/meta-data",
    "seed_directory_url": "http://10.1.10.130:8000/automation/test/"
  }
}
```

Complete End-To-End Ansible + CustomISO API

POST /api/v1/provision

```
{
  "bmc_address": "10.0.10.141",
  "bmc_username": "root",
  "bmc_password": "password",
  "os_type": "ubuntu",
  "os_version": "24.04.4",
  "arch": "amd64",
  "variant": "ipxe-uefi",
  "hostname": "test",
  "username": "default",
  "password": "default",
  "ipv4_address": "10.1.10.142",
  "ipv4_gateway": "10.1.10.1",
  "ipv4_netmask": "255.255.255.224",
  "ipv6_address": "1234:1234:1234:1234::1234",
  "ipv6_gateway": "fe80::1234:1234:1234:1234",
  "ipv6_cidr": "64",
  "dns_servers": "8.8.8.8,8.8.4.4,1.1.1.1,2001:4860:4860::8888",
  "raid": false
}
```

Response:

```
{
  "success": true,
  "body": {
    "job_id": test
  }
}
```

Job query

```
GET /status?job_id=test
```

Example Response:

```
{
  "job_id": "test",
  "lifecycle_stage": "BOOTING",
  "stage_description": "ISO/iPXE compilation sequence complete. Waiting for
system boot.",
  "ansible_staging": {
    "status": "none",
    "rc": null
  }
}
```